Full Syllabus of Agentic AI

Serially arranged learning path from fundamentals to deployment

1. Course Overview

Agentic AI is the design of AI systems that can reason, plan, use tools, remember context, and execute multi-step tasks with limited human intervention. This syllabus moves from core concepts to advanced agent workflows and production deployment.

2. Prerequisites

- Python basics: variables, loops, functions, classes, modules
- Basic understanding of APIs and JSON
- Familiarity with machine learning and LLMs is helpful, but not mandatory
- Comfort with Git, command line, and notebook environments

3. Learning Outcomes

- Explain what an AI agent is and how it differs from a normal chatbot
- Design prompt, tool, memory, and planning components for agent systems
- Build RAG-based and tool-using agents
- Evaluate agent quality, cost, latency, and safety
- Deploy an agentic application with logging, monitoring, and guardrails

4. Serial Syllabus

4.1 Foundations of AI and LLMs

- What is AI, ML, NLP, and generative AI
- Transformer architecture in simple terms
- Tokens, context window, embeddings, and attention
- Capabilities and limitations of large language models

4.2 Introduction to Agentic AI

- Definition of an AI agent
- Chatbot vs assistant vs agent
- Agent loop: observe, think, act, and reflect
- Single-agent and multi-agent systems

4.3 Problem Framing for Agents

- Choosing tasks suitable for agents
- Decomposing a goal into steps
- Human-in-the-loop vs autonomous execution
- Success criteria and failure conditions

4.4 Prompt Engineering for Agents

- System prompts, developer prompts, and user prompts

- Role prompting, constraint prompting, and output formatting
- Few-shot examples for reliable behavior
- Prompt chaining and prompt templates

4.5 Tool Usage and Function Calling

- Why agents need tools
- APIs, database queries, search tools, calculators, and file tools
- Function calling and structured outputs
- Tool selection, validation, and error handling

4.6 Memory in Agentic Systems

- Short-term vs long-term memory
- Conversation memory and task memory
- Summarization memory
- Vector databases for memory retrieval

4.7 Planning and Reasoning

- Planning strategies: linear, hierarchical, and reflective
- Task decomposition and subgoals
- Self-check, critique, and re-planning
- Reasoning traces, structured reasoning, and tool-aware reasoning

4.8 Retrieval-Augmented Generation (RAG)

- Why RAG is important for grounded answers
- Document ingestion, chunking, and indexing
- Embeddings and similarity search
- Retrieval, reranking, and answer generation

4.9 Agentic RAG

- Combining retrieval with action execution
- Query rewriting and multi-hop retrieval
- Context enrichment before response generation
- When to use RAG vs when to use a tool

4.10 Workflow Orchestration

- State machines and graph-based workflows
- Conditional branching and looping
- Sequential pipelines vs dynamic execution graphs
- Introduction to orchestration frameworks such as LangChain and LangGraph

4.11 Multi-Agent Systems

- Coordinator, planner, executor, and critic agents
- Specialization and delegation
- Communication between agents
- Consensus, arbitration, and conflict resolution

4.12 Agent Evaluation and Testing

- Correctness, groundedness, and task completion rate
- Latency, cost, and token usage

- Automated test cases and golden datasets
- Adversarial testing and red-teaming

4.13 Safety, Security, and Guardrails

- Prompt injection and jailbreak risks
- Data privacy and sensitive information handling
- Tool sandboxing and permission control
- Output filtering, policy checks, and safe fallback behavior

4.14 Deployment and Observability

- Packaging an agentic app with Streamlit or a web backend
- Logging, tracing, and debugging agent steps
- Monitoring quality regressions
- Scaling considerations, caching, and rate limits

4.15 Advanced Topics

- Model routing and multi-model systems
- Structured outputs with JSON schemas
- Memory compaction and long-horizon agents
- Voice agents, browser agents, and code agents

5. Practical Labs

1. Lab 1: Build a simple LLM chatbot with prompt templates
2. Lab 2: Add tool calling for calculator, search, or database access
3. Lab 3: Create a RAG pipeline over PDF documents
4. Lab 4: Add memory and conversation summarization
5. Lab 5: Build a graph-based agent workflow with branching
6. Lab 6: Create a multi-agent task system with planner and executor roles
7. Lab 7: Add logging, guardrails, and evaluation metrics

6. Capstone Project Ideas

- Customer support agent that answers from company documents and can create tickets
- Research assistant that searches documents, summarizes, and produces citations
- Sales assistant that qualifies leads, updates CRM data, and drafts follow-up emails
- Personal productivity agent that manages tasks, reminders, and summaries

7. Assessment Pattern

- Theory quizzes on LLMs, RAG, agents, memory, and safety
- Hands-on assignments after each module
- Mid-project review for architecture and reliability
- Final capstone demonstration with evaluation report

8. Suggested Duration

A balanced schedule is 6 to 10 weeks, depending on depth and project work.

9. Recommended Tools

- Python, Jupyter, VS Code
- OpenAI / Gemini / Anthropic or other LLM APIs
- LangChain, LangGraph, or similar orchestration libraries
- FAISS, Chroma, Pinecone, or other vector databases
- Streamlit, FastAPI, or Flask for deployment